

Report IX

Cubic spline interpolation

Author: Patryk Kuna

1. Introduction

Cubic spline interpolation is a numerical method used to approximate a function based on a set of known data points. Unlike global polynomial interpolation, which uses a single high-degree polynomial, spline interpolation constructs a set of low-degree polynomials defined on several subintervals.

For a given set of nodes (x_i, y_i) , cubic spline ensures:

- continuity of the function
- continuity of the first derivative
- continuity of the second derivative

In this project, natural cubic splines are used, meaning that the second derivatives at the boundaries are equal to zero:

$$\frac{d^2y}{dx^2} \Big|_{x=x_0} = 0, \quad \frac{d^2y}{dx^2} \Big|_{x=x_n} = 0$$

The interpolation is performed for the function:

$$y(x) = \frac{1}{1+x^2}$$

defined on the interval:

$$x \in [-5, 5]$$

The goal of this exercise is to analyze the accuracy of a cubic spline interpolation and investigate how the number of nodes affects the quality of approximation.

2. Description of the Numerical Task

The interpolation procedure was carried out according to the following steps:

- generation of equidistant interpolation nodes
- computation of function values $y_i = y(x_i)$
- construction of a tridiagonal system of equations
- solving the system to obtain second derivatives at nodes
- evaluation of the spline on a dense grid
- computation of the interpolation error

The following cases were analyzed:

- $n = 5$
- $n = 10$
- $n = 50$

The interpolation error was defined as:

$$\Delta(x) = y(x) - s(x)$$

Additionally, numerical second derivatives obtained from the spline were compared with exact analytical values.

3. Numerical Method

Cubic spline interpolation reduces to solving a tridiagonal system of linear equations:

$$A \cdot M = d$$

where:

- M_i - second derivatives at nodes
- A - tridiagonal matrix
- d - right-hand side vector

The matrix has the tridiagonal structure:

- diagonal: 2 (except boundaries = 1)
- subdiagonal: μ_i
- superdiagonal: λ_i

The coefficients are defined as:

$$h_i = x_i - x_{i-1}$$

$$\lambda_i = \frac{h_{i+1}}{h_i + h_{i+1}},$$

$$\mu_i = 1 - \lambda_i$$

$$d_i = \frac{6}{h_i + h_{i+1}} \left(\frac{y_{i+1} - y_i}{h_{i+1}} - \frac{y_i - y_{i-1}}{h_i} \right)$$

After solving the system, the spline on each interval $[x_i, x_{i+1}]$ is constructed as:

$$s_i(x) = \frac{M_i(x_{i+1} - x)^3}{6h} + \frac{M_{i+1}(x - x_i)^3}{6h} + A_i(x - x_i) + B_i$$

This formulation guarantees smoothness and local stability of the interpolation.

4. Implementation

The program was written in C++ and followed closely the algorithm provided in the instruction file, using numerical library LAPACK for solving the tridiagonal system.

Main steps of the implementation:

1. Data generation

- equidistant nodes:

$$x_i = x_{\min} + i \cdot \Delta x$$

2. Function evaluation

- computation of $y(x)$ for the whole range

3. Matrix construction

- arrays:
 - dl - subdiagonal
 - d - diagonal
 - du - superdiagonal
- right-hand side vector b

4. Solving the system

- LAPACKE routine: LAPACKE_dgtsv()

5. Spline evaluation

- scanning interval with dense grid (199 points)
- identifying interval
- computing spline values

6. Data export

- exact function
- spline values
- interpolation error
- second derivatives

Data was saved to text files for plotting.

5. Results

Case 1: $n = 5$

- basic approximation of the function
- visible deviation from exact solution
- large interpolation error

Case 2: $n = 10$

- significantly improved accuracy
- good agreement near the boundaries of the interval
- moderate errors near the center region

Case 3: $n = 50$

- very high accuracy
- spline almost indistinguishable from exact function
- error is very small across the entire domain

Interpolation error

- decreases with increasing number of nodes
- remains smooth and stable
- no large oscillations observed

Second derivatives

- numerical values agree well with the exact ones at boundaries
- slight differences appear in the central region

Figure 1. $y(x)$ and $s(x)$ for $n = 5$

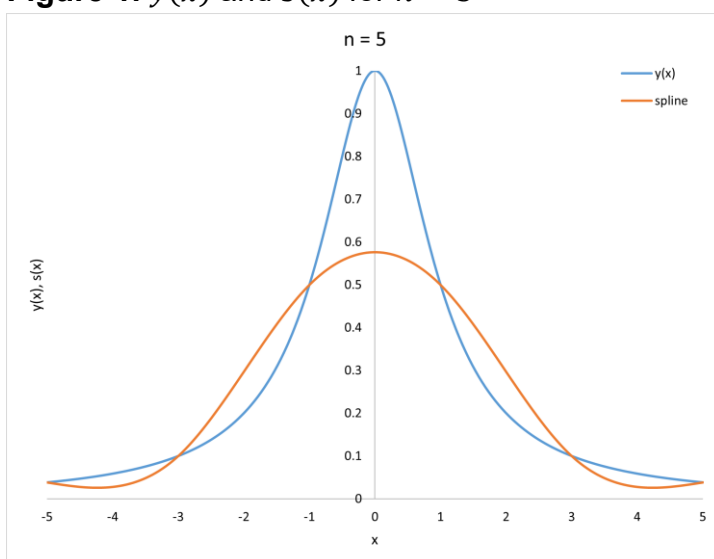


Figure 2. $y(x)$ and $s(x)$ for $n = 10$

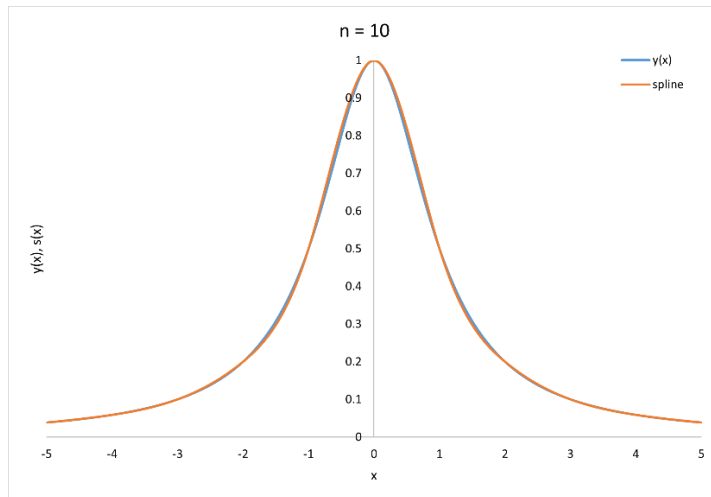


Figure 3. $y(x)$ and $s(x)$ for $n = 50$

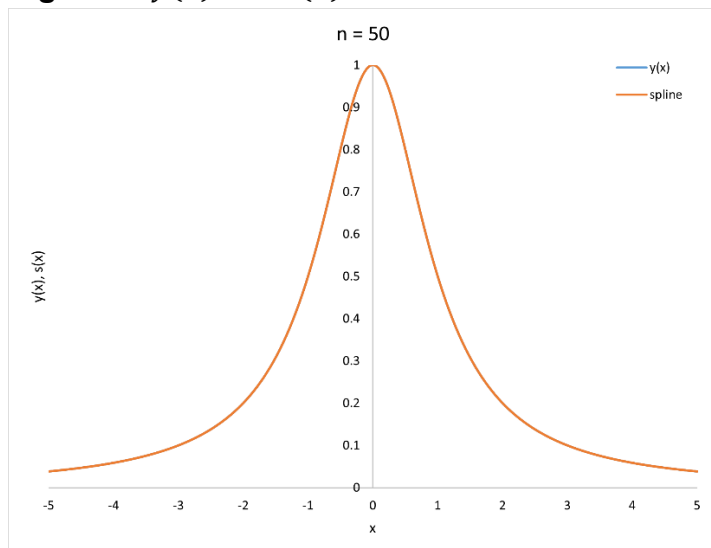
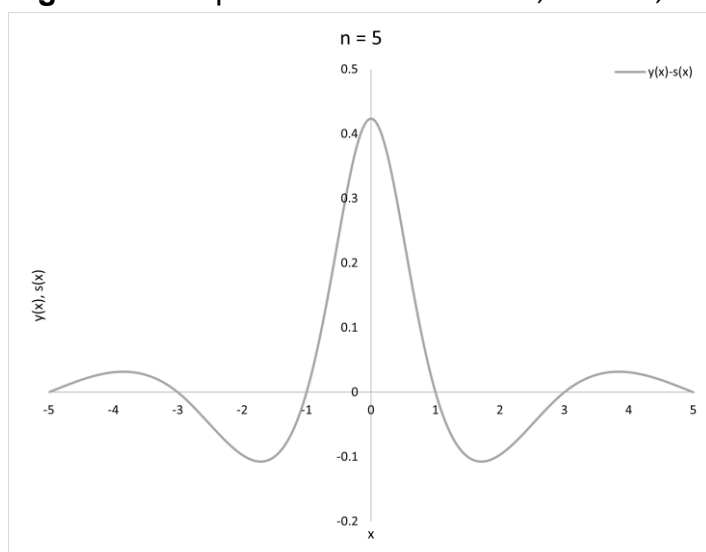


Figure 4. Interpolation error for $n = 5$, $n = 10$, $n = 50$



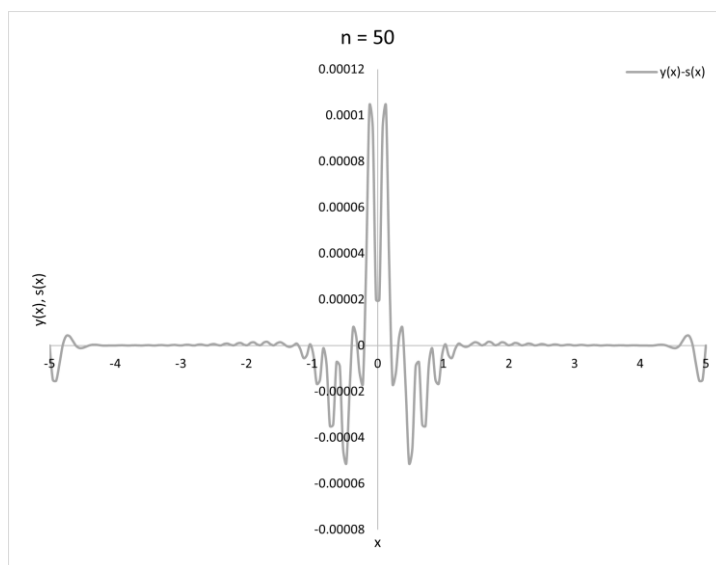
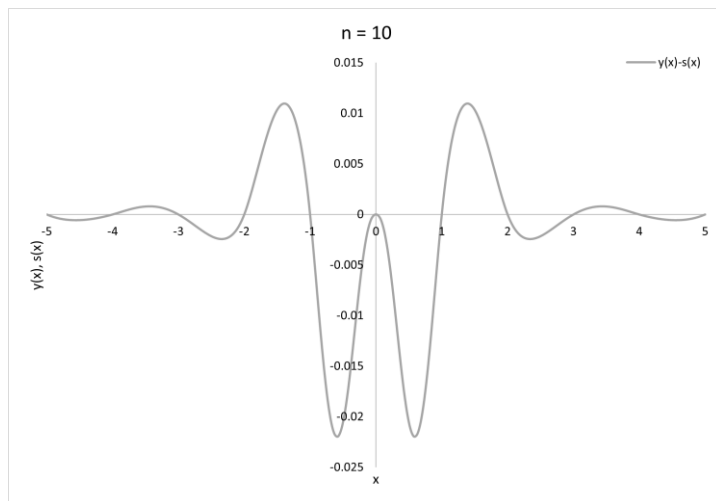
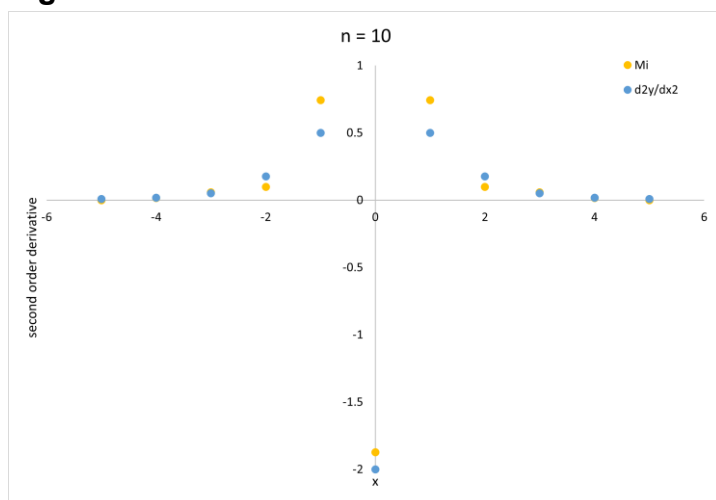


Figure 5. Second order derivative at nodes for $n = 10$



6. Conclusions

Cubic spline interpolation provides a stable and accurate method for function approximation.

The following observations were made:

- increasing number of nodes improves accuracy
- spline interpolation avoids large oscillations
- method remains stable even for large n
- interpolation error decreases smoothly across the domain
- no Rungee phenomenon was observed

Compared to polynomial interpolation, cubic splines offer significantly better numerical behavior, especially near the boundaries.

Overall, cubic spline interpolation is an efficient and reliable method for approximating smooth functions.